

Arquitectura Multi-App con PostgreSQL Compartido

Problema Actual:

Actualmente tienes PostgreSQL dentro del docker-compose de Ecodisseny. Para añadir Django Oscar, esto no es escalable.

✓ Solución Recomendada: PostgreSQL Compartido

Estructura Propuesta:

```

└─ /root/
   └─ docker-services/                # 🗂️ Servicios compartidos
      ├── docker-compose.yml          # PostgreSQL + pgAdmin + Redis
      └── .env                        # Variables de BD compartidas
   └─ ecodisseny_dj_pg/               # 🌱 App 1: Ecodisseny
      ├── docker-compose.yml          # Solo Django web (puerto 8000)
      └── .env                        # Variables específicas
   └─ oscar_shop/                     # 🛒 App 2: Oscar Shop
      ├── docker-compose.yml          # Solo Django web (puerto 8001)
      └── .env                        # Variables específicas
```

MIGRACIÓN PASO A PASO

1. Crear Servicios Compartidos

```
# Crear estructura
mkdir -p /root/docker-services
cd /root/docker-services

# Docker Compose para servicios compartidos
cat > docker-compose.yml << 'EOF'
version: '3.8'

services:
  postgres:
    image: postgres:15
    restart: always
    container_name: shared_postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres_master_password_2024
      POSTGRES_DB: postgres
```

```
volumes:
  - postgres_shared_data:/var/lib/postgresql/data
  - ./init-databases.sql:/docker-entrypoint-initdb.d/init-databases.sql
ports:
  - "5432:5432"
networks:
  - shared_network
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 10s
  timeout: 5s
  retries: 5

pgadmin:
  image: dpage/pgadmin4:latest
  restart: always
  container_name: shared_pgadmin
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@ecodisseny.local
    PGADMIN_DEFAULT_PASSWORD: admin2024
  ports:
    - "8080:80"
  volumes:
    - pgadmin_data:/var/lib/pgadmin
  networks:
    - shared_network
  depends_on:
    postgres:
      condition: service_healthy

redis:
  image: redis:7-alpine
  restart: always
  container_name: shared_redis
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  networks:
    - shared_network

volumes:
  postgres_shared_data:
  pgadmin_data:
  redis_data:

networks:
  shared_network:
    name: shared_services
    driver: bridge

EOF

# Script para crear bases de datos
cat > init-databases.sql << 'EOF'
```

```
-- Crear bases de datos para cada aplicación
CREATE DATABASE ecodisseny_db;
CREATE DATABASE oscar_shop_db;

-- Crear usuarios específicos
CREATE USER ecodisseny_user WITH ENCRYPTED PASSWORD
'ecodisseny_password123';
CREATE USER oscar_user WITH ENCRYPTED PASSWORD 'oscar_password123';

-- Dar permisos
GRANT ALL PRIVILEGES ON DATABASE ecodisseny_db TO ecodisseny_user;
GRANT ALL PRIVILEGES ON DATABASE oscar_shop_db TO oscar_user;

-- Configurar permisos por defecto
ALTER DATABASE ecodisseny_db OWNER TO ecodisseny_user;
ALTER DATABASE oscar_shop_db OWNER TO oscar_user;
EOF

# Iniciar servicios compartidos
docker-compose up -d
```

2. Migrar Ecodisseny a la Nueva Arquitectura

```
cd /root/ecodisseny_dj_pg

# Backup actual de la BD
docker-compose exec db pg_dump -U ecodisseny_user ecodisseny_db >
backup_antes_migracion.sql

# Nuevo docker-compose.yml (solo Django)
cat > docker-compose.yml << 'EOF'
version: '3.8'

services:
  web:
    build: .
    restart: always
    container_name: ecodisseny_web
    ports:
      - "8000:8000"
    volumes:
      - ./app
      - static_volume:/app/staticfiles
      - media_volume:/app/media
    environment:
      - DEBUG=False
      - ALLOWED_HOSTS=161.97.147.142,app.arasmu.net
      - DB_HOST=shared_postgres
      - DB_NAME=ecodisseny_db
      - DB_USER=ecodisseny_user
      - DB_PASSWORD=ecodisseny_password123
```

```

- DB_PORT=5432
- STATIC_ROOT=/app/staticfiles
# Redis para cache/sesiones
- REDIS_URL=redis://shared_redis:6379/1
networks:
- shared_services
command: >
  sh -c "python manage.py migrate &&
        python manage.py collectstatic --noinput &&
        gunicorn ecodisseny.wsgi:application --bind 0.0.0.0:8000"

volumes:
  static_volume:
  media_volume:

networks:
  shared_services:
    external: true
EOF

# Parar el stack actual
docker-compose down

# Restaurar datos en PostgreSQL compartido
cat backup_antes_migracion.sql | docker exec -i shared_postgres psql -U
ecodisseny_user -d ecodisseny_db

# Iniciar nueva configuración
docker-compose up -d

```

3. Crear Estructura para Oscar Shop

```

# Crear proyecto Oscar
mkdir -p /root/oscar_shop
cd /root/oscar_shop

# Crear docker-compose.yml para Oscar
cat > docker-compose.yml << 'EOF'
version: '3.8'

services:
  web:
    build: .
    restart: always
    container_name: oscar_web
    ports:
      - "8001:8000" # Puerto diferente
    volumes:
      - ./app
      - static_volume:/app/staticfiles
      - media_volume:/app/media

```

```
environment:
  - DEBUG=False
  - ALLOWED_HOSTS=161.97.147.142,tienda.arasmu.net
  - DB_HOST=shared_postgres
  - DB_NAME=oscar_shop_db
  - DB_USER=oscar_user
  - DB_PASSWORD=oscar_password123
  - DB_PORT=5432
  - STATIC_ROOT=/app/staticfiles
  # Redis para cache/sesiones
  - REDIS_URL=redis://shared_redis:6379/2
networks:
  - shared_services
command: >
  sh -c "python manage.py migrate &&
        python manage.py collectstatic --noinput &&
        gunicorn oscar_shop.wsgi:application --bind 0.0.0.0:8000"

volumes:
  static_volume:
  media_volume:

networks:
  shared_services:
    external: true
EOF

# Dockerfile para Oscar
cat > Dockerfile << 'EOF'
FROM python:3.12-slim

# Variables de entorno
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Instalar dependencias del sistema
RUN apt-get update && apt-get install -y \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Crear directorio de trabajo
WORKDIR /app

# Copiar requirements y instalar
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar proyecto
COPY . .

# Crear usuario no root
RUN useradd --create-home --shell /bin/bash oscar
RUN chown -R oscar:oscar /app
```

```
USER oscar
```

```
EXPOSE 8000
```

```
EOF
```

VENTAJAS DE ESTA ARQUITECTURA

Recursos Optimizados:

- ♦ ANTES (PostgreSQL por app):
 - └ Ecodisseny: Django + PostgreSQL = ~350MB RAM
 - └ Oscar: Django + PostgreSQL = ~350MB RAM
 - └ Total: ~700MB RAM
- ♦ DESPUÉS (PostgreSQL compartido):
 - └ PostgreSQL compartido: ~200MB RAM
 - └ Ecodisseny Django: ~150MB RAM
 - └ Oscar Django: ~150MB RAM
 - └ Total: ~500MB RAM (30% menos)

Gestión de Datos:

- ✓ Backup centralizado: Un solo PostgreSQL
- ✓ Administración: pgAdmin para todas las apps
- ✓ Escalabilidad: Fácil añadir nuevas apps
- ✓ Cache compartido: Redis para todas las apps
- ✓ Monitoreo: Un solo punto de fallo en BD

Gestión de Servicios:

```
# Comandos centralizados:  
cd /root/docker-services && docker-compose logs postgres  
cd /root/docker-services && docker-compose restart postgres  
  
# Por aplicación:  
cd /root/ecodisseny_dj_pg && docker-compose restart web  
cd /root/oscar_shop && docker-compose restart web
```

SCRIPTS DE GESTIÓN

Script de Monitoreo Multi-App

```

cat > /root/scripts/monitor_multi_app.sh << 'EOF'
#!/bin/bash
echo "🏠 ESTADO MULTI-APP ARCHITECTURE"
echo "===== "

echo "📁 SERVICIOS COMPARTIDOS:"
cd /root/docker-services
docker-compose ps

echo ""
echo "🌱 ECODISSENY:"
cd /root/ecodisseny_dj_pg
docker-compose ps

echo ""
echo "🛒 OSCAR SHOP:"
cd /root/oscar_shop
if [ -f docker-compose.yml ]; then
    docker-compose ps
else
    echo "No configurado aún"
fi

echo ""
echo "🌐 CONECTIVIDAD:"
echo "Ecodisseny (8000): $(curl -s -o /dev/null -w "%{http_code}"
http://localhost:8000 2>/dev/null || echo 'Error')"
echo "Oscar (8001): $(curl -s -o /dev/null -w "%{http_code}"
http://localhost:8001 2>/dev/null || echo 'Error')"
echo "pgAdmin (8080): $(curl -s -o /dev/null -w "%{http_code}"
http://localhost:8080 2>/dev/null || echo 'Error')"

echo ""
echo "🗄️ BASES DE DATOS:"
docker exec shared_postgres psql -U postgres -c "\l" | grep -E "
(ecodisseny|oscar)"
EOF

chmod +x /root/scripts/monitor_multi_app.sh

```

Script de Backup Multi-App

```

cat > /root/scripts/backup_multi_app.sh << 'EOF'
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/root/backups/multi_app"

mkdir -p $BACKUP_DIR

echo "📁 Backup Multi-App ($DATE)"

```

```
# Backup PostgreSQL completo
echo "📁 Backup bases de datos..."
docker exec shared_postgres pg_dumpall -U postgres >
$BACKUP_DIR/all_databases_$DATE.sql
gzip $BACKUP_DIR/all_databases_$DATE.sql

# Backup individual por app
docker exec shared_postgres pg_dump -U ecodisseny_user ecodisseny_db >
$BACKUP_DIR/ecodisseny_$DATE.sql
docker exec shared_postgres pg_dump -U oscar_user oscar_shop_db >
$BACKUP_DIR/oscar_$DATE.sql 2>/dev/null || echo "Oscar DB no existe aún"

# Backup configuraciones
tar -czf $BACKUP_DIR/docker_configs_$DATE.tar.gz /root/docker-services
/root/ecodisseny_dj_pg/docker-compose.yml /root/oscar_shop/docker-
compose.yml 2>/dev/null

echo "✅ Backup completado en $BACKUP_DIR"
EOF

chmod +x /root/scripts/backup_multi_app.sh
```

🎯 DECISIÓN RECOMENDADA

✅ Sí, migra a PostgreSQL compartido porque:

1. 🎯 **Eficiencia:** Menos RAM, menos complejidad
2. ⚡ **Escalabilidad:** Fácil añadir Oscar y futuras apps
3. 🛠️ **Mantenimiento:** Backups y administración centralizados
4. 💰 **Costo:** Mejor uso de recursos del VPS
5. 🏢 **Arquitectura:** Más profesional y estándar

📋 Plan de Migración:

1. ✅ Crear servicios compartidos (PostgreSQL + pgAdmin + Redis)
2. ✅ Migrar Ecodisseny a nueva arquitectura
3. ✅ Verificar funcionamiento
4. ✅ Crear estructura Oscar Shop
5. ✅ Configurar Nginx para múltiples puertos
6. ✅ Scripts de monitoreo y backup centralizados

¿Procedo con la implementación de esta migración? 🚀